

SYNTHÈSE SUR L'ÉVOLUTION ET L'ÉTAT DE L'ART DES MÉTHODES DE SYNCHRONISATION LOGICIELLE ET MATÉRIELLE

Ilian Benaoudia
Zakaria Bouslah

Table des matières

1	Introduction	3
2	Évolution Historique de la Synchronisation par Décennie	3
2.1	Années 1960 : Premiers Algorithmes d'Exclusion Mutuelle et Sémaphores	3
2.2	Années 1970 : Moniteurs, Algorithme de la Boulangerie et Primitives Matérielles	4
2.3	Années 1980 : Nouveaux Algorithmes et Premiers Concepts de Synchronisation Non Bloquante	5
2.4	Années 1990 : Fondations Théoriques et Avancées en Synchronisation Non Bloquante	5
2.5	Années 2000 : Multiprocesseurs, Scalabilité et Innovations . .	6
2.6	Années 2010 : Avancées sur la Mémoire Transactionnelle Matérielle et Optimisations des Verrous	7
2.7	Années 2020 : Tendances Actuelles et Recherches en Cours . .	8
3	Panorama des Techniques de Synchronisation	9
3.1	Mutex et Verrous	9
3.2	Sémaphores et Variables de Condition	9
3.3	Algorithmes Non Bloquants	9
3.4	Mémoire Transactionnelle	10
3.5	Approches Spécialisées	10

4	Travaux Récents et Domaines de Recherche Émergents	10
5	Conclusion	11

1 Introduction

La synchronisation en informatique consiste à coordonner les processus ou threads concurrents afin qu'ils puissent accéder à des ressources partagées en toute sécurité, sans conflit. Une synchronisation efficace est essentielle à la fois pour assurer la correction des programmes et pour exploiter pleinement les performances du matériel parallèle. Les systèmes d'exploitation modernes et les applications reposent sur une variété de primitives de synchronisation — depuis les verrous simples (mutex, sémaphores) jusqu'à des algorithmes sans verrou sophistiqués — afin d'éviter les conditions de concurrence et d'assurer la cohérence des données.

Le choix de la méthode de synchronisation appropriée implique souvent un compromis entre la performance (débit, latence) et la complexité. Ce document propose une vue d'ensemble approfondie de l'état de l'art des méthodes de synchronisation logicielle et matérielle, incluant :

- une revue historique par décennie des principales évolutions (algorithmes de Dekker, Peterson, introduction des instructions atomiques, mémoire transactionnelle, etc.) ;
- une présentation des catégories majeures de synchronisation (mutex, sémaphores, spinlocks, algorithmes sans verrou, mémoire transactionnelle) ;
- un accent particulier sur les recherches récentes et les développements en cours.

2 Évolution Historique de la Synchronisation par Décennie

2.1 Années 1960 : Premiers Algorithmes d'Exclusion Mutuelle et Sémaphores

La décennie 1960 marque l'émergence des premières solutions au problème de l'exclusion mutuelle — à savoir, empêcher que deux processus n'entrent simultanément dans une section critique. L'un des premiers algorithmes logiciels pour l'exclusion mutuelle est celui de **Dekker** [2] (1965), permettant à deux processus de partager une ressource sans conflit, sans recourir à du matériel spécifique.

Parallèlement, **Edsger W. Dijkstra** [3] introduit le concept de *sémasphores*, avec les opérations *P* (prendre) et *V* (signaler), comme primitives de plus haut niveau pour gérer l'accès concurrent aux ressources. Dans un article de 1965, Dijkstra propose une solution simple au problème d'exclusion mutuelle à l'aide de variables partagées.

Ces premières approches sont remarquables car elles réalisent l'exclusion mutuelle uniquement à l'aide d'opérations de lecture et écriture, sans primitives atomiques spécifiques — une nécessité à une époque où les architectures ne proposaient pas encore d'instructions atomiques. Cependant, ces solutions étaient souvent limitées à deux processus, difficiles à généraliser à un nombre plus important, et pouvaient s'avérer inefficaces.

Les sémaphores trouvent rapidement une application dans les systèmes d'exploitation, offrant un moyen pratique d'éviter les conditions de course par synchronisation des événements.

2.2 Années 1970 : Moniteurs, Algorithme de la Boulangerie et Primitives Matérielles

Durant les années 1970, la synchronisation connaît des avancées majeures tant du point de vue de l'abstraction logicielle que du support matériel.

Les moniteurs sont introduits par C.A.R. Hoare et Per Brinch Hansen comme constructions de synchronisation de haut niveau intégrées aux langages de programmation. Un moniteur encapsule des variables partagées avec leurs procédures d'accès, garantissant que seul un processus à la fois peut exécuter une procédure du moniteur. L'approche moniteur, avec ses variables de condition associées, simplifie énormément la programmation concurrente, et influence directement des langages ultérieurs comme Java (avec les blocs *synchronized*).

Au niveau algorithmique, **l'algorithme de la boulangerie** de Leslie Lamport [8] (1974) généralise les idées de Dekker à un nombre quelconque de processus. Chaque processus obtient un numéro (comme un ticket à la boulangerie), et la priorité d'accès à la section critique est donnée à celui ayant le plus petit numéro. Cette solution logicielle garantit à la fois l'équité et l'absence d'attente indéfinie, sans recourir à des primitives matérielles.

Par ailleurs, les architectes de matériel introduisent des primitives atomiques destinées à la synchronisation efficace. Le **IBM System/360** propose dès 1964 l'instruction *Test-and-Set*, permettant la réalisation de spin-

locks simples. Plus tard, le *Compare-and-Swap* (CAS), introduit sur le System/370, fournit une primitive encore plus puissante : elle permet de vérifier et modifier une valeur mémoire de manière atomique.

Ces instructions atomiques facilitent grandement la conception de verrous efficaces, en supprimant la nécessité d’algorithmes logiciels complexes comme ceux de Dekker ou Lamport, et posent les bases pour les futures générations d’algorithmes de synchronisation.

2.3 Années 1980 : Nouveaux Algorithmes et Premiers Concepts de Synchronisation Non Bloquante

Durant les années 1980, la synchronisation continue d’évoluer avec l’optimisation des méthodes classiques et l’émergence des premiers concepts non bloquants.

En 1981, l’**algorithme de Peterson** [10] propose une solution d’exclusion mutuelle compacte et à la fois simple et correcte pour deux processus. Utilisant deux indicateurs et une variable de tour, il parvient à éviter les conditions de course tout en conservant un code facile à comprendre et à généraliser.

Dans les systèmes multiprocesseurs, l’usage des *spinlocks* se généralise. Toutefois, les problèmes de performance liés à une forte contention apparaissent, menant à l’invention du *Test-and-Test-and-Set*, réduisant la charge sur le bus mémoire.

Par ailleurs, les premiers travaux sur les *algorithmes non bloquants* voient le jour. La pile sans verrou de **Treiber** [11] (1986), basée sur Compare-and-Swap, constitue l’une des premières applications pratiques d’une structure de données *lock-free*. Ces concepts pionniers introduisent l’idée qu’un blocage d’un thread ne doit pas empêcher le progrès des autres, ouvrant la voie à des systèmes concurrents plus résilients.

2.4 Années 1990 : Fondations Théoriques et Avancées en Synchronisation Non Bloquante

Les années 1990 marquent une étape majeure avec la formalisation rigoureuse de la correction des objets concurrents. **Herlihy et Wing**[7] introduisent la notion de *linéarisabilité*, qui définit la cohérence d’un objet concurrent par rapport à un ordre global atomique.

Maurice Herlihy expose dans ses travaux de 1991 la hiérarchie des primitives de synchronisation selon leur puissance à permettre une synchronisation *wait-free*. Il montre notamment que des primitives simples comme *Test-and-Set* ne suffisent pas pour garantir l'attente bornée, tandis que *Compare-and-Swap* (CAS) est universellement puissant.

Sur le plan pratique, de nouvelles structures à forte scalabilité apparaissent. Le *verrou MCS* (Mellor-Crummey et Scott, 1991) introduit une file d'attente pour réduire la contention sur les verrous à grande échelle, garantissant l'accès FIFO à la section critique.

En parallèle, le concept de *mémoire transactionnelle* (Transactional Memory) est proposé par Herlihy et Moss [6] (1993). L'objectif est de simplifier la programmation concurrente en permettant d'exécuter des blocs critiques comme des transactions atomiques — soit l'ensemble des modifications est validé, soit elles sont annulées.

Enfin, les *structures de données lock-free* gagnent en maturité, à l'image de la file d'attente lock-free de Michael et Scott (1996), constituant une avancée importante pour les systèmes hautement concurrents.

2.5 Années 2000 : Multiprocesseurs, Scalabilité et Innovations

Avec l'essor des multiprocesseurs à partir des années 2000, la synchronisation doit être repensée pour mieux exploiter les architectures à grande échelle.

Les *futex* (*fast user-space mutex*) apparaissent sous Linux vers 2002. Cette technique permet de combiner le verrouillage en espace utilisateur et l'endormissement en espace noyau uniquement en cas de contention, réduisant drastiquement les coûts.

Parallèlement, le *Read-Copy Update* (RCU) est intégré au noyau Linux. Cette approche optimise les accès en lecture, permettant à de multiples lecteurs de coexister sans blocage, tandis que les mises à jour sont différées jusqu'à ce qu'aucun lecteur n'accède à l'ancienne version.

Du côté de la mémoire transactionnelle, de nombreux efforts visent à la rendre réaliste en logiciel (STM) et à proposer des prototypes matériels (HTM). Le concept de *Transactional Lock Elision* — consistant à tenter une exécution transactionnelle avant de verrouiller — devient une stratégie prometteuse.

De plus, les algorithmes *lock-free* et *wait-free* continuent à se perfectionner, notamment grâce à de meilleures méthodes de gestion de la mémoire (ex : pointeurs à risque, récupération basée sur les époques).

2.6 Années 2010 : Avancées sur la Mémoire Transactionnelle Matérielle et Optimisations des Verrous

Durant les années 2010, les processeurs multi-cœurs deviennent la norme même pour les dispositifs mobiles, tandis que les serveurs atteignent plusieurs dizaines de cœurs. Les techniques de synchronisation — tant classiques qu’innovantes — doivent être repensées pour répondre à cette explosion du parallélisme.

Un développement majeur est l’apparition de la **mémoire transactionnelle matérielle** (HTM) dans les processeurs commerciaux. En 2013, **Intel** introduit les *Transactional Synchronization Extensions* (TSX) dans l’architecture Haswell, permettant d’exécuter des blocs de code de manière transactionnelle : en cas d’absence de conflit, les modifications sont engagées atomiquement ; sinon, la transaction est abandonnée et doit être retentée ou basculée vers une solution de secours classique à base de verrou.

Bien que prometteuse, l’HTM n’a pas remplacé les verrous traditionnels, notamment à cause de ses limites (ex : taille restreinte des transactions, interruptions externes causant des **aborts**). Néanmoins, elle est exploitée dans certains systèmes pour l’**élision de verrous** (lock elision) : tenter une exécution transactionnelle rapide avant de tomber en repli sur un verrou réel en cas d’échec.

Parallèlement, la conception des verrous continue à s’affiner :

- **Verrous adaptatifs** : combinant attente active et endormissement en fonction de la durée estimée d’attente.
- **Verrous conscients de l’architecture NUMA** : réduisant les transferts de données entre noeuds de mémoire pour de meilleures performances.
- **Verrous hiérarchiques** : adaptant la méthodologie de prise de verrou selon le niveau de contention observé.

Des approches hybrides — combinant verrous, transactions et délégation — se développent à travers des techniques telles que le *flat combining*, où les threads délèguent temporairement leurs opérations à un unique combiné pour réduire la contention.

Enfin, les efforts se poursuivent pour rendre les algorithmes *lock-free* et *wait-free* plus efficaces en pratique, avec notamment l'introduction de stratégies d'entraide entre threads (helping) pour garantir une progression bornée sans sacrifier la performance.

2.7 Années 2020 : Tendances Actuelles et Recherches en Cours

Les années 2020 s'inscrivent dans la continuité des développements précédents, tout en adressant de nouveaux défis liés à la montée en flèche du parallélisme massif et à l'apparition de nouvelles mémoires persistantes.

L'accent est mis sur l'**amélioration de la scalabilité** des méthodes de synchronisation classiques. Les verrous traditionnels, bien que robustes, deviennent insuffisants sur des architectures à centaines de coeurs : les stratégies de *flat combining* et de *délégation* continuent à être explorées pour réduire la contention et le rebond de cache.

Les évolutions du matériel, notamment l'émergence de la **mémoire persistante** (PMEM), imposent de nouvelles contraintes. Il ne suffit plus d'assurer la sécurité des accès concurrents : il faut aussi garantir la cohérence en cas de panne, en intégrant des *barrières de persistance* aux protocoles de synchronisation. De nombreuses recherches portent sur l'adaptation des techniques classiques à ce contexte (journalisation, copie-sur-écriture, transactions persistantes).

Par ailleurs, l'essor des architectures hétérogènes (combinaisons CPU-GPU) et du calcul distribué soulève de nouveaux besoins de synchronisation inter-domaines, où les latences et les modèles de mémoire diffèrent.

Enfin, la **vérification formelle** des algorithmes de synchronisation gagne en importance. L'usage de model checking, de preuves assistées par ordinateur et de techniques de vérification automatique se généralise afin de garantir la correction des systèmes dans un contexte de complexité croissante.

Les tendances contemporaines mettent donc l'accent sur la résilience, l'efficacité à grande échelle et l'assurance de la correction — autant d'axes qui continueront à façonner les méthodes de synchronisation dans les prochaines décennies.

3 Panorama des Techniques de Synchronisation

3.1 Mutex et Verrous

Les mutex (*mutual exclusion locks*) sont l'une des primitives les plus fondamentales en synchronisation. Ils permettent d'assurer qu'un seul thread à la fois accède à une ressource partagée. Les verrous simples basés sur *Test-and-Set* ou *Compare-and-Swap* sont suffisants dans les systèmes à faible contention, mais deviennent inefficaces sous forte charge.

Des variantes plus sophistiquées incluent :

- **Spinlocks optimisés** : par exemple le *Test-and-Test-and-Set* pour limiter le trafic mémoire.
- **Verrous à file d'attente** : MCS, CLH, permettant d'obtenir un accès FIFO et de réduire la contention sur les bus de mémoire.
- **Verrous adaptatifs** : combinant attente active et endormissement dynamique en fonction de la charge.

3.2 Sémaphores et Variables de Condition

Les sémaphores généralisent l'exclusion mutuelle en gérant l'accès à un nombre fini de ressources. Un sémaphore binaire agit comme un mutex, tandis qu'un sémaphore comptant permet la synchronisation d'accès multiples.

Les variables de condition, souvent associées à un mutex, permettent à un thread de s'endormir jusqu'à ce qu'une certaine condition soit remplie, facilitant l'implémentation d'attentes complexes.

3.3 Algorithmes Non Bloquants

Les algorithmes *lock-free* et *wait-free* à l'aide de primitives atomiques comme CAS offrent des garanties de progrès supérieures :

- **Lock-free** : garantit qu'au moins un thread fait des progrès.
- **Wait-free** : garantit que chaque thread fait des progrès dans un nombre borné d'étapes.

Des structures de données telles que les piles de Treiber, les files de Michael-Scott, et les listes chaînées lock-free, illustrent ces concepts. Le développement de méthodes efficaces pour la gestion de la mémoire (e.g., techniques à base d'époques) est crucial pour leur mise en œuvre correcte.

3.4 Mémoire Transactionnelle

La *mémoire transactionnelle* propose d'exécuter des sections critiques sous forme de transactions atomiques. Elle peut être :

- **STM** (Software Transactional Memory) : implémentée en logiciel.
- **HTM** (Hardware Transactional Memory) : supportée directement par le matériel, comme Intel TSX.

La mémoire transactionnelle simplifie la programmation concurrente mais pose des problèmes en termes de gestion des conflits, d'aborts répétés, et de limites structurelles.

3.5 Approches Spécialisées

Certaines techniques visent à réduire encore plus la contention :

- **Read-Copy-Update** (RCU) : permet des lectures ultra-rapides sans verrou.
- **Flat Combining** : agrégation des opérations concurrentes par un seul thread combiné.
- **Delegation et Server-based Synchronization** : externalisation des opérations critiques à des threads spécifiques.

4 Travaux Récents et Domaines de Recherche Émergents

Les dernières années ont vu l'émergence de nombreux travaux visant à repousser les limites des techniques classiques de synchronisation.

- **Synchrobench** (Gramoli [4], 2015) : ce benchmark compare de manière systématique diverses structures de données concurrentes sur différentes plateformes, mettant en évidence les compromis entre verrouillage, non-blocage et contention.
- **Flat Combining** (Hendler [5] et al., 2010) : cette approche propose de combiner les opérations de plusieurs threads sous la responsabilité d'un unique thread leader, afin de réduire la pression sur les verrous et amortir le coût des accès concurrents.
- **Read-Log-Update** (RLU) : cette technique généralise les idées de Read-Copy-Update à des structures de données plus complexes, en

autorisant des lectures ultra-rapides combinées à des journaux d'écriture atomiques.

- **Synchronisation persistante** : de nombreuses recherches (ex : Calciu [1] et al., 2018) explorent comment adapter la synchronisation à la mémoire persistante (NVRAM), avec des structures comme les transactions persistantes et les verrous durables.
- **Algorithmes à progression garantie** : des travaux comme ceux de Kogan, Petrank, Timnat proposent de nouvelles approches pratiques pour obtenir la progression wait-free ou obstruction-free dans des contextes réalistes.
- **Analyses comparatives inter-OS** : des études récentes (Okoe & Toyemade [9], 2024) analysent l'efficacité des primitives de synchronisation sur différents systèmes d'exploitation modernes, mettant en évidence l'impact des choix d'implémentation sur les performances multicœurs.

De manière générale, la synchronisation contemporaine s'oriente vers des solutions :

- plus résilientes aux pannes (mémoire persistante),
- plus scalables sur des systèmes massivement parallèles,
- plus faciles à vérifier formellement (linéarisabilité, modèles formels),
- plus adaptées aux contextes spécifiques (GPU, cloud, edge computing).

Ces directions de recherche illustrent la vitalité et la complexité croissante des problèmes de synchronisation dans les systèmes informatiques modernes.

5 Conclusion

La synchronisation est un domaine fondamental de l'informatique, en constante évolution depuis ses premières formulations dans les années 1960. L'histoire montre une progression continue, des algorithmes purement logiciels aux primitives atomiques matérielles, jusqu'aux solutions adaptées aux architectures massivement parallèles et à la mémoire persistante.

Les techniques modernes doivent concilier performances, scalabilité, simplicité d'usage et garantie formelle de correction. Les verrous classiques, les approches non bloquantes, la mémoire transactionnelle, ainsi que les méthodes spécialisées comme RCU ou flat combining, montrent la richesse des stratégies envisageables.

Les recherches actuelles mettent également en lumière la nécessité d'adapter les mécanismes de synchronisation aux nouvelles contraintes du calcul distribué, des systèmes hétérogènes et des mémoires non volatiles. À l'avenir, l'enjeu sera de développer des méthodes toujours plus performantes, résilientes et formellement vérifiables.

Références

- [1] Irina Calciu et al. Persistent memory : Challenges and opportunities. In *Proceedings of the International Symposium on Memory Systems (MEMSYS)*, 2018.
- [2] Th. J. Dekker. A note on a problem in concurrent programming control. *Communications of the ACM*, 8(9) :569, 1965.
- [3] Edsger W. Dijkstra. Solution of a problem in concurrent programming control. *Communications of the ACM*, 1965.
- [4] Vincent Gramoli. More than you ever wanted to know about synchronization : Synchrobench, measuring the impact of the synchronization on concurrent algorithms. *Proceedings of the 20th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2015.
- [5] Danny Hendler, Itai Incze, Nir Shavit, and Moran Tzafrir. Flat combining and the synchronization-parallelism tradeoff. In *Proceedings of the 22nd ACM Symposium on Parallelism in Algorithms and Architectures*, 2010.
- [6] Maurice Herlihy and J. Eliot B. Moss. Transactional memory : Architectural support for lock-free data structures. In *Proceedings of the 20th Annual International Symposium on Computer Architecture*, pages 289–300. IEEE Computer Society, 1993.
- [7] Maurice Herlihy and Jeannette Wing. Linearizability : A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems*, 1990.
- [8] Leslie Lamport. A new solution of dijkstra’s concurrent programming problem. In *Communications of the ACM*, 1974.
- [9] Michael Okoe and S. Toyemade. Comparative analysis of synchronization primitives in modern operating systems, 2024. Preprint.
- [10] Gary L. Peterson. Myths about the mutual exclusion problem. *Information Processing Letters*, 12(3) :115–116, 1981.
- [11] R. K. Treiber. Systems programming : Coping with parallelism. *Technical Report RJ 5118, IBM Almaden Research Center*, 1986.